

Pregunta 2: Evolución del Control de Cambios

Liderazgo, Conflicto y Seguridad Psicológica. Contexto: Usted es el nuevo Agile Coach / Líder Técnico de un equipo. Nota que en las revisiones de código (Pull Requests), el Arquitecto Principal (un desarrollador muy senior y brillante) utiliza comentarios sarcásticos y destructivos hacia los desarrolladores junior, tales como *¿En serio vas a usar este patrón? Es de principiantes*. Los juniors han dejado de proponer ideas en las retrospectivas y la velocidad de entrega ha caído.

Diagnostique la situación utilizando el concepto de Seguridad Psicológica de Amy Edmondson. ¿Qué impacto tiene esto en el rendimiento del equipo a largo plazo?

Diseñe una estrategia de intervención utilizando principios de Liderazgo de Servicio y Comunicación No Violenta. ¿Cómo abordaría el conflicto con el Arquitecto Principal en privado y cómo reestructuraría el proceso de revisión de código para objetivar el feedback?

Proponga una dinámica para la próxima Retrospectiva que ayude a reconstruir la confianza del equipo.

Respuesta:

Diagnóstico de la situación

La situación muestra una falta de seguridad psicológica en el equipo. Según Amy Edmondson, existe seguridad psicológica cuando las personas pueden expresar ideas, hacer preguntas, admitir errores o plantear dudas sin miedo a ser ridiculizadas o castigadas.

Los comentarios sarcásticos del Arquitecto Principal, como *“¿En serio vas a usar este patrón? Es de principiantes”*, no critican solamente el código: también descalifican públicamente a la persona. Como consecuencia, los desarrolladores junior han dejado de participar en las retrospectivas y de proponer ideas.

Esto indica que el equipo está actuando de manera defensiva. Los integrantes prefieren guardar silencio para evitar humillaciones, aunque tengan información importante sobre errores, riesgos o posibles mejoras.

Impacto a largo plazo

La falta de seguridad psicológica afecta directamente el rendimiento del equipo:

- Disminuye la participación y la creatividad.
- Los desarrolladores dejan de hacer preguntas y de compartir errores.
- Se ocultan problemas técnicos hasta que son más difíciles de solucionar.

- Aumenta el retrabajo y disminuye la calidad del software.
- Se debilita la colaboración entre personas senior y junior.
- Puede aumentar la rotación y el agotamiento del personal.
- La velocidad disminuye porque las revisiones se convierten en discusiones defensivas.

Por lo tanto, aunque el Arquitecto tenga mucho conocimiento técnico, su comportamiento está generando un costo para todo el equipo. Un equipo puede tener excelentes profesionales, pero rendir mal si sus integrantes sienten que no pueden hablar con libertad.

Estrategia de intervención

1. Conversación privada con el Arquitecto

El líder debe reunirse con el Arquitecto Principal en privado, describiendo conductas concretas y su impacto, sin atacarlo personalmente.

Podría decirle:

“En varias revisiones de código has utilizado expresiones como ‘¿En serio vas a usar este patrón? ¿Es de principiantes?’. He observado que después de esos comentarios los desarrolladores junior participan menos y evitan proponer alternativas. Necesitamos mantener un alto nivel técnico, pero también crear un ambiente donde las personas puedan aprender y expresar sus ideas.”

Después, se debe aclarar la expectativa:

“Te pido que critiques el código y expliques los riesgos técnicos, pero sin utilizar sarcasmos ni descalificaciones personales. Por ejemplo, en lugar de decir que un patrón es de principiantes, podrías explicar qué problema puede generar y proponer una alternativa.”

Esto aplica el liderazgo de servicio porque el líder no busca imponer autoridad, sino ayudar a que el equipo trabaje mejor y que el conocimiento del Arquitecto se convierta en mentoría.

2. Aplicar Comunicación No Violenta

La Comunicación No Violenta utiliza cuatro elementos: **observación, sentimiento, necesidad y petición.**

Un ejemplo de conversación sería:

- **Observación:** “En las últimas revisiones se han escrito comentarios sarcásticos sobre las propuestas de algunos desarrolladores junior.”

- **Sentimiento:** “Me preocupa y me frustra observar que ahora participan menos en las reuniones.”
- **Necesidad:** “Necesitamos respeto, aprendizaje y libertad para discutir alternativas técnicas.”
- **Petición:** “¿Estarías dispuesto a formular tus observaciones sobre el código de manera técnica y respetuosa, y a explicar tus razones antes de rechazar una propuesta?”

Este enfoque evita frases como “eres una persona tóxica” y se concentra en comportamientos que pueden modificarse. La petición también debe ser concreta y observable.

3. Establecer reglas para las revisiones de código

El equipo debe acordar reglas claras:

- Criticar el código, no a la persona.
- Explicar el motivo técnico de cada comentario importante.
- Diferenciar entre un error crítico y una preferencia personal.
- Usar preguntas como “¿consideraste esta alternativa?” en lugar de afirmaciones despectivas.
- Reconocer también los aspectos correctos del trabajo.
- Permitir que el autor responda y explique sus decisiones.
- Escalar los desacuerdos técnicos mediante una discusión, no mediante sarcasmo.

Por ejemplo, se puede reemplazar:

“Ese patrón es de principiantes.”

Por:

“Me preocupa que este patrón aumente el acoplamiento entre los módulos. ¿Podemos comparar esta opción con el patrón X y revisar cuál se adapta mejor a este caso?”

4. Recuperar la participación del equipo

En las siguientes retrospectivas, el Agile Coach puede utilizar dinámicas que den voz a todos:

- Recoger primero ideas por escrito y de forma anónima.
- Dar a cada persona un tiempo breve para hablar.
- Preguntar directamente a los miembros junior por sus opiniones, sin obligarlos a exponerse.
- Separar la generación de ideas de la evaluación.

- Reconocer públicamente aportes útiles.
- Revisar periódicamente si las personas se sienten seguras para participar.

El líder también debe modelar el comportamiento esperado: admitir sus propios errores, pedir opiniones y agradecer cuando alguien señala un riesgo.

5. Dar seguimiento

La intervención debe evaluarse durante varias semanas mediante indicadores como:

- Mayor participación de los juniors en retrospectivas.
- Menor cantidad de comentarios personales o sarcásticos.
- Más propuestas técnicas en los Pull Requests.
- Reducción del retrabajo.
- Mejora en la percepción de respeto y seguridad del equipo.
- Recuperación progresiva de la velocidad sin sacrificar calidad.

Si el comportamiento destructivo continúa después de la conversación y de las reglas acordadas, debe aplicarse una medida formal de liderazgo, porque la experiencia técnica no justifica dañar al equipo.

¿Cómo abordaría el conflicto con el Arquitecto Principal en privado y cómo reestructuraría el proceso de revisión de código para objetivar el feedback?

La conversación con el Arquitecto Principal se plantea en privado, nunca frente al equipo, siguiendo los principios de Liderazgo de Servicio y Comunicación No Violenta, evitando cualquier tono acusatorio que active una postura defensiva. La conversación se organiza en cuatro momentos:

- Observación: se presentan hechos concretos y verificables, por ejemplo, dos o tres comentarios reales dejados en Pull Requests, sin interpretarlos ni etiquetarlos como "grosería" o "mal carácter".
- Impacto: se muestra la consecuencia observable en el equipo, como la caída en la participación de los junior en las retrospectivas y la reducción en la velocidad de entrega, presentada como un dato medible y no como un juicio de valor.
- Necesidad: se explica la necesidad compartida detrás de la conversación, un ambiente donde todo el equipo pueda aportar ideas sin temor, lo cual también favorece al propio Arquitecto, ya que un equipo silenciado deja de detectar errores.
- Petición: se cierra con una petición concreta y accionable, no con una advertencia genérica, como acordar juntos un nuevo formato de retroalimentación en las revisiones de código.

Desde el Liderazgo de Servicio, el objetivo de esta conversación no es sancionar al Arquitecto, sino eliminar el impedimento que su forma de comunicarse representa para el equipo, entendiendo que su comportamiento puede originarse en presión o en un perfeccionismo técnico mal canalizado, más que en una intención deliberada de dañar.

En paralelo, se plantea reestructurar el proceso de revisión de código en tres frentes:

- Adoptar una guía de estilo y estándares de codificación documentados y acordados por el equipo, de modo que las observaciones se justifiquen contra un criterio objetivo y no contra la preferencia personal del revisor.
- Delegar en herramientas automatizadas (linters, análisis estático, Quality Gates de integración continua) la detección de problemas básicos de estilo o calidad, reduciendo el espacio para comentarios subjetivos o sarcásticos.
- Rotar la responsabilidad de revisión entre distintos miembros del equipo, evitando que los junior dependan siempre de la aprobación de una sola persona.

Proponga una dinámica para la próxima Retrospectiva que ayude a reconstruir la confianza del equipo.

Se plantea una Retrospectiva bajo el formato del Velero (Sailboat Retrospective), complementada con una ronda inicial de apreciación anónima, pensada para bajar la barrera de exposición que sienten los desarrolladores junior después de haber sido desalentados a opinar.

La sesión se abre recitando la Directriz Principal (Prime Directive) de las retrospectivas: la afirmación explícita de que se asume que todos hicieron el mejor trabajo posible dadas las circunstancias que conocían en su momento, estableciendo desde el inicio un marco de no culpabilización.

A continuación, cada integrante escribe de forma anónima una idea o comentario que en algún momento no se atrevió a compartir en una retrospectiva o revisión de código anterior. El facilitador las lee en voz alta sin atribuir autoría, abriendo la discusión sin señalar a nadie.

Posteriormente, se aplica la metáfora del velero, identificando en conjunto:

- Los vientos: lo que está impulsando al equipo.
- Las anclas: lo que lo está frenando, incluyendo explícitamente la dinámica de las revisiones de código.
- Las rocas: los riesgos futuros si la situación no se corrige.
- La isla: la meta compartida de un equipo colaborativo y psicológicamente seguro.

Nombrar la dinámica destructiva como una "ancla" identificada por todo el equipo, y no solo señalada por el líder técnico, distribuye la responsabilidad de la mejora y evita que el Arquitecto Principal sienta que está siendo señalado de forma individual.

La sesión cierra con un acuerdo concreto y visible para todos, como la adopción de la nueva guía de revisión de código, dejando constancia de que el cambio nace de una decisión colectiva del equipo y no de una imposición jerárquica.